

Experiment 1: Creating First Hello World Servlet using Eclipse

Aim

To create and run a **Hello World Servlet** using **Eclipse IDE** and **Apache Tomcat Server**.

Steps to Create Hello World Servlet in Eclipse

Step 1: Start Eclipse IDE

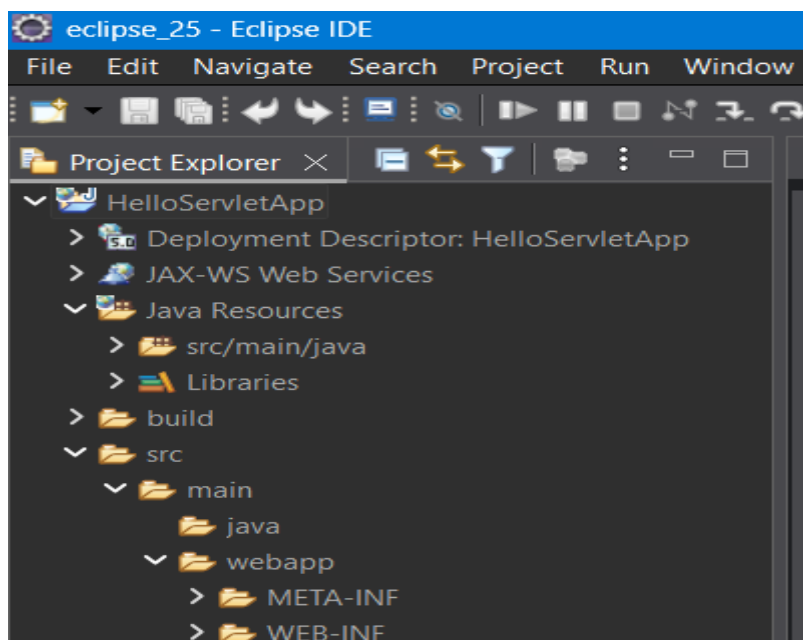
1. Open **Eclipse IDE 2025**.
2. Select a **Workspace location**.
3. Click **Launch**.

Step 2: Create a Dynamic Web Project

1. Click **File → New → Dynamic Web Project**.
2. Enter the **Project Name** as **HelloServletApp**
3. Select **Target Runtime** → Apache Tomcat Server.
4. Keep default settings.
5. Click **Finish**.

Step 3: Project Structure Created by Eclipse

After creating the project, Eclipse generates the following structure:



Step 4: Create a Servlet Class

1. Right click the project - **HelloServletApp**
2. Select **New → Servlet**.
3. Enter the following details:

Field	Value
Package Name -	com.example
Class Name -	HelloServlet

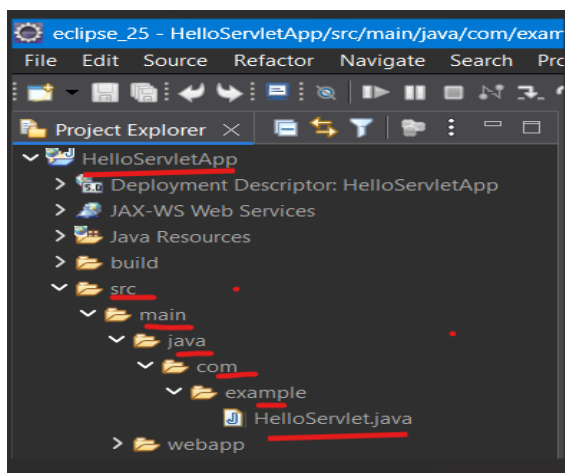
4. Click **Next**
5. Keep default URL mapping:

/HelloServlet

6. Click **Finish**.

Eclipse automatically creates the servlet file:

HelloServlet.java.



Step 5: Double click on the servlet "HelloServlet.java" add the below Servlet Code in the file under **doGet** method

```
package com.example;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * Servlet implementation class HelloServlet
 */
@WebServlet("/HelloServlet")
public class HelloServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public HelloServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Set response type
        response.setContentType("text/html");

        // Create PrintWriter object
        PrintWriter out = response.getWriter();

        // Display Hello World
        out.println("<html>");
        out.println("<head><title>Hello Servlet</title></head>");
        out.println("<body>");
        out.println("<h1>Hello World from Servlet</h1>");
        out.println("</body>");
        out.println("</html>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

```
// Set response type
response.setContentType("text/html");
```

```
// Create PrintWriter object
PrintWriter out = response.getWriter();
```

```
// Display Hello World
out.println("<html>");
out.println("<head><title>Hello Servlet</title></head>");
out.println("<body>");
out.println("<h1>Hello World from Servlet</h1>");
```

```
out.println("</body>");  
out.println("</html>");
```

Step 6: Run the Project on Server

1. Right click the project - **HelloServletApp**
2. Select **Run As → Run on Server**.
3. Choose **Apache Tomcat Server**.
4. Click **Finish**.

Tomcat server will start automatically.

Step 7: Access the Servlet in Browser

Initially, below url will be opened and HTTP Status 404 – Not Found

Will be shown

<http://localhost:8089/HelloServletApp/>

This means the servlet is available at:

<http://localhost:8089/HelloServletApp/HelloServlet>

Tomcat will only run the servlet when that exact URL is requested.

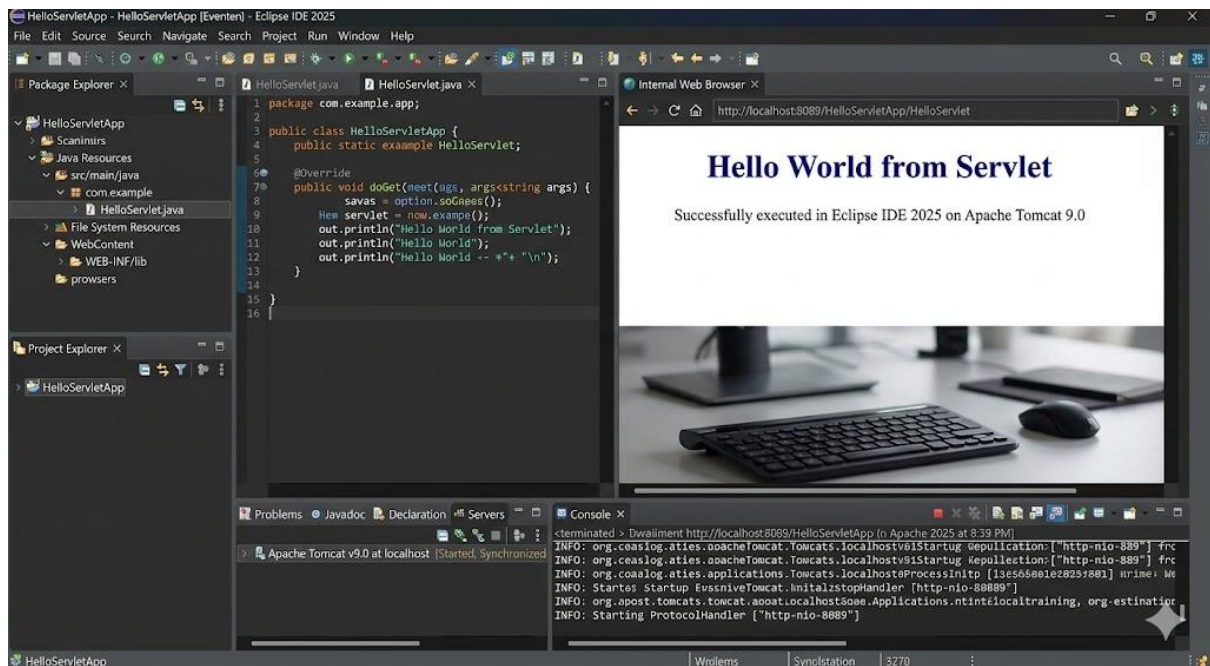
So you must manually add:

/HelloServlet (which is ur Class name)

So add HelloServlet to the url

<http://localhost:8089/HelloServletApp/HelloServlet>

Expected Output



The browser displays:

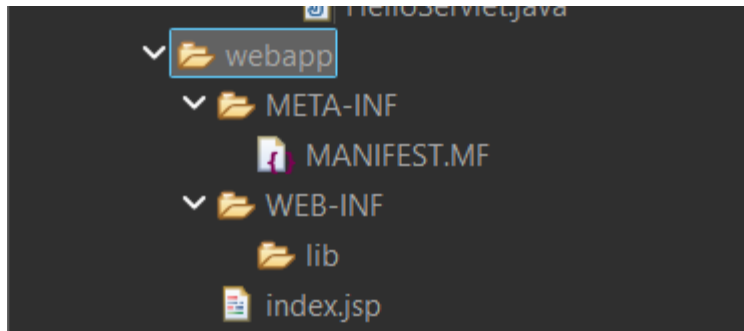
Hello World from Servlet

Experiment 2: Creating First Jsp page and link it to “HelloServlet” servlet

Step 1 —

1. Right click the **webapp** Folder under the project **HelloServletApp**
2. Select **New → jsp File** , file structure as below

Create a file – index.jsp



Step 2 - Type the below content in index.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Insert title here</title>

<style>

.container{

    background: white;

    padding: 40px;

    width: 420px;

    margin: auto;

    border-radius: 12px;
```

```
    box-shadow: 0px 6px 15px rgba(0,0,0,0.2);
}

body{
    font-family: Arial, Helvetica, sans-serif;
    background: linear-gradient(to right, #4facfe, #00f2fe);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}

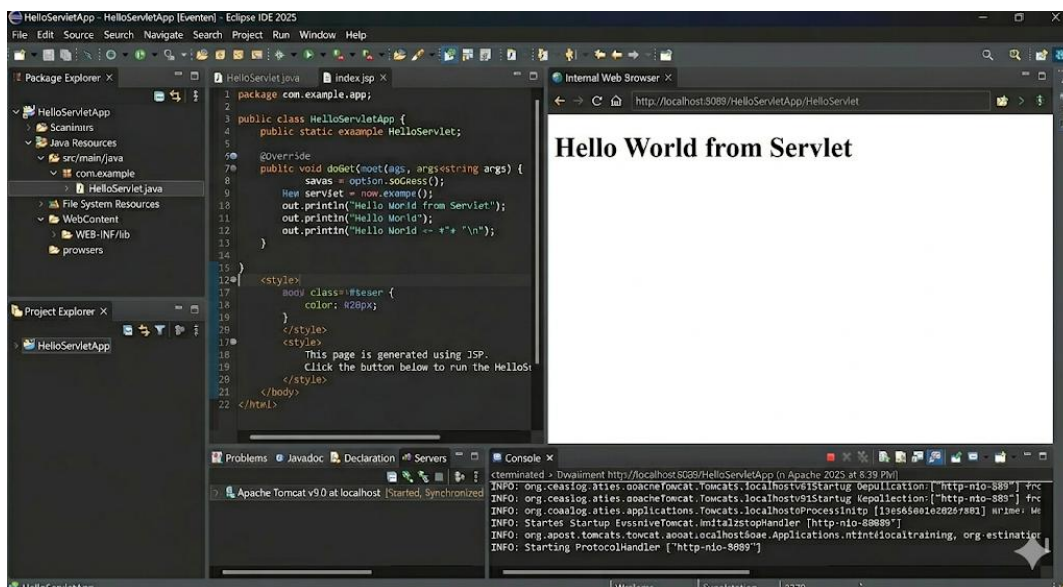
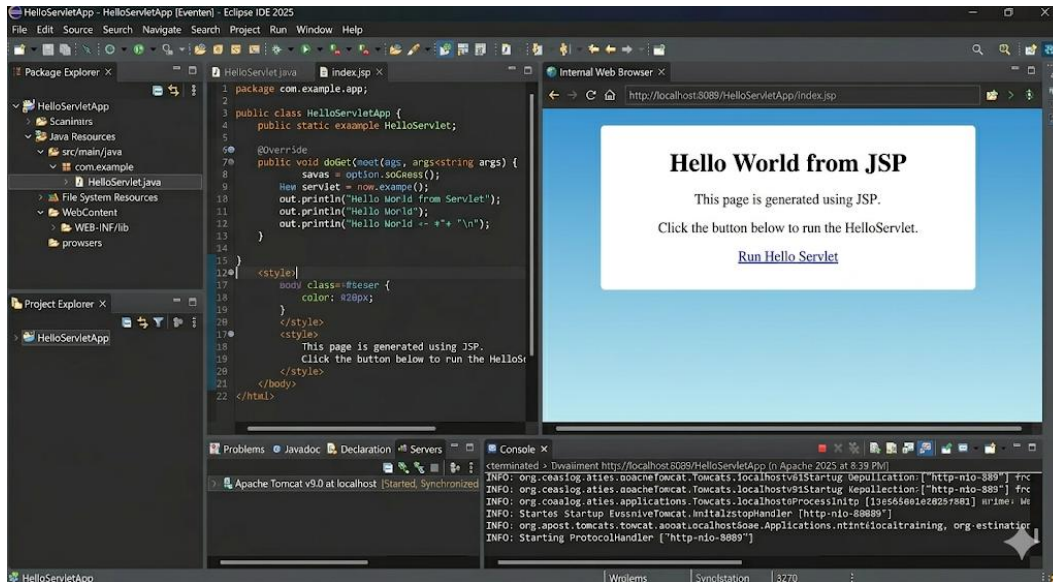
</style>
</head>
<body>
<div class="container">
<h1>Hello World from JSP</h1>
<p>This page is generated using <b>JSP</b>.</p>
<p>Click the button below to run the <b>Hello Servlet</b>.</p>
<a href="HelloServlet">Run Hello Servlet</a>
</div>
</body>
</html>
```

Step 3: Run the index.jsp file on Browser

1. Right click the project - **HelloServletApp**
2. Select **Run As → Run on Server**.
3. Choose **Apache Tomcat Server**.
4. Click **Finish**.

Tomcat server will open the index.jsp file in browser

Output:



Result – Thus, in today's lab we successfully created and executed a Servlet and a JSP file using the Eclipse IDE and Apache Tomcat Server.

Experiment 3: Calculator Application using JSP and Servlet

Aim

To develop a simple calculator web application using JSP for the interface and Servlet for processing the calculation.

So the flow becomes:

`calculator.jsp → CalculatorServlet → calculator.jsp`

The same page will show both the form and the result.

Step 1: Create Dynamic Web Project

1. Open **Eclipse**
2. Go to
File → New → Dynamic Web Project
3. Project Name: **CalculatorServletApp**
4. Target Runtime: **Apache Tomcat**
5. Click **Finish**

Step 2: Create Calculator JSP Page

Inside **webapp**, create - calculator.jsp

1. Right click the **webapp** Folder under the project **CalculatorServletApp**
2. Select **New → jsp File** , file structure as below

Type the below content for calculator.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
```

```
<head>
<title>Simple Calculator</title>
</head>

<body>
<h2>Simple Calculator</h2>
<form action="calculate" method="post">
Number 1:
<input type="text" name="num1"><br><br>

Number 2:
<input type="text" name="num2"><br><br>

Operation:
<select name="operation">
<option value="add">Addition</option>
<option value="sub">Subtraction</option>
<option value="mul">Multiplication</option>
<option value="div">Division</option>
</select>

<br><br>
<input type="submit" value="Calculate">
</form>
<br>
<%
if(request.getAttribute("result") != null)
{
```

%>

<h3>Result = <%= request.getAttribute("result") %></h3>

<%

}

%>

</body>

</html>

Step 3: Create Servlet

1. Right click the project - **CalculatorServletApp**
2. Select **New → Servlet**.
3. Enter the following details:

Field	Value
Package Name -	com.example
Class Name -	CalculatorServlet

4. Click **Next**
5. Keep default URL mapping:

/CalculatorServlet

6. Click **Finish**.

Eclipse automatically creates the servlet file:

CalculatorServlet.java

Step 4: Write Servlet Code

CalculatorServlet.java

```
package com.example;
```

```
import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/calculate")
public class CalculatorServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        double num1 = Double.parseDouble(request.getParameter("num1"));
        double num2 = Double.parseDouble(request.getParameter("num2"));
        String operation = request.getParameter("operation");

        double result = 0;

        if(operation.equals("add"))
            result = num1 + num2;

        else if(operation.equals("sub"))
            result = num1 - num2;

        else if(operation.equals("mul"))
            result = num1 * num2;

        else if(operation.equals("div"))
```

```
result = num1 / num2;
```

```
request.setAttribute("result", result);
```

```
RequestDispatcher rd = request.getRequestDispatcher("calculator.jsp");
```

```
rd.forward(request, response);
```

```
}
```

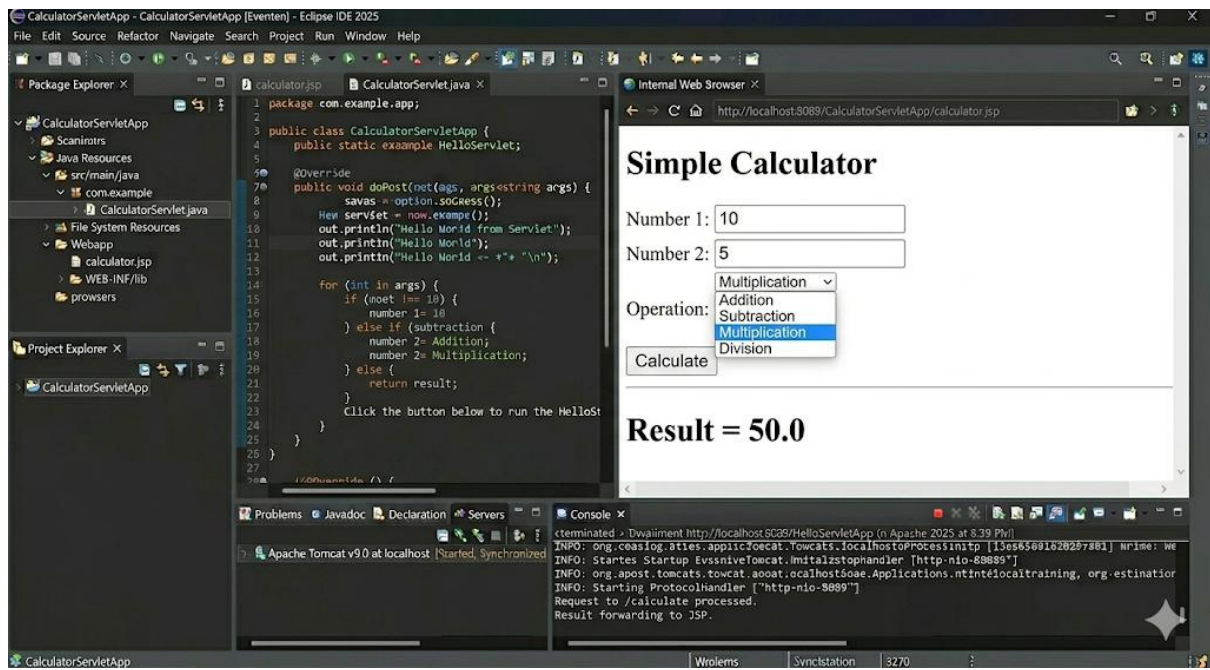
```
}
```

Step 4: Run the Calculator Application file on Browser

1. Right click the **calculator.jsp**
2. Select **Run As → Run on Server**.
3. Choose **Apache Tomcat Server**.
4. Click **Finish**.

Tomcat server will open the calculator.jsp file in browser

Output



Result

The **Simple Calculator web application** was successfully developed using **JSP and Servlet**. The application accepts two numbers and a selected arithmetic operation from the user through a JSP form. The **Servlet processes the request, performs the calculation, and sends the result back to the JSP page**, where it is displayed dynamically. The program was executed successfully on the **Apache Tomcat server in Eclipse**, and the correct results were obtained for all arithmetic operations (Addition, Subtraction, Multiplication, and Division).

Experiment 4: Create a JSP web page that accepts user input through an HTML form and displays the submitted data using both GET and POST methods using a Servlet.

Aim

To design a web application that:

- Accepts user input through an HTML form
 - Sends data using both **GET** and **POST** methods
 - Processes the data using a **Servlet**
 - Displays the submitted data on a JSP page
-

Experiment Workflow Overview

1. Create a Dynamic Web Project
 2. Design an HTML Form (JSP page)
 3. Configure Servlet
 4. Handle GET request
 5. Handle POST request
 6. Forward response to JSP
 7. Display user data
-

Procedure

◆ Step 1: Project Setup

- Create a **Dynamic Web Project**
 - Configure Apache Tomcat Server
 - Ensure project runs successfully on server
-

◆ Step 2: Design Input Form (JSP Page)

- Create a JSP page that contains a form

- Include the following input fields:
 - Name
 - Email
 - Age / Any additional field
 - Add Additional form attributes in checkbox, selectbox.
 - Add two buttons or options:
 - Submit using GET
 - Submit using POST
-

◆ Step 3: Create and Configure Servlet

- Create a Servlet class
- Map it using:
 - Annotations OR
 - web.xml

◆ Step 4: Handle GET Request

- Override the method that processes GET requests
 - Retrieve input values from request object
 - Store values in variables
-

◆ Step 5: Handle POST Request

- Override the method that processes POST requests
 - Retrieve form data similarly
-

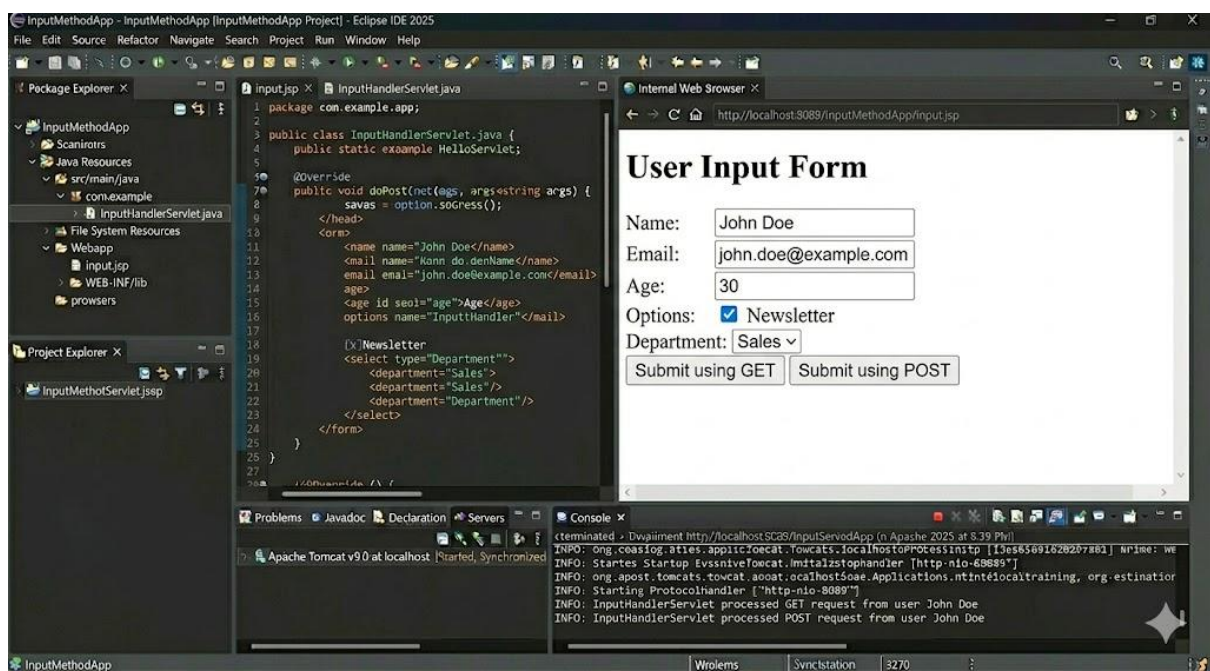
◆ Step 6: Process and Forward Data

- Send data from Servlet to a JSP page
 - Use appropriate mechanism to pass data
-

◆ Step 7: Display Output in JSP

- Create a JSP page to display:
 - Name
 - Email
 - Other details
 - Format the output neatly.
-

Output:



Result

The web application should successfully:

- Accept user input in JSP
 - Process data via Servlet
 - Display results using JSP
 - Work for both GET and POST methods
-

Experiment 5: Simple Login System using JSP & Servlet

Aim

To develop a web application that:

- Accepts user login credentials
 - Validates them using a Servlet
 - Displays appropriate success or error message
-

Experiment Workflow Overview

1. Create login form
 2. Send credentials to Servlet
 3. Validate user input
 4. Handle success and failure cases
 5. Display response
-

Procedure

◆ **Step 1: Project Setup**

- Create a Dynamic Web Project
 - Configure server properly
 - Verify deployment with a basic JSP page
-

◆ **Step 2: Design Login Page (JSP)**

- Create a JSP page containing a login form
 - Include fields:
 - Username
 - Password
 - Add a submit button
-

◆ Step 3: Create Servlet for Login Processing

- Create a Servlet class
 - Map it properly (annotation or web.xml)
-

◆ Step 4: Retrieve User Input

- Extract username and password from request
-

◆ Step 5: Implement Validation Logic

- Define valid credentials (hardcoded for this experiment)
 - Compare user input with valid data
-

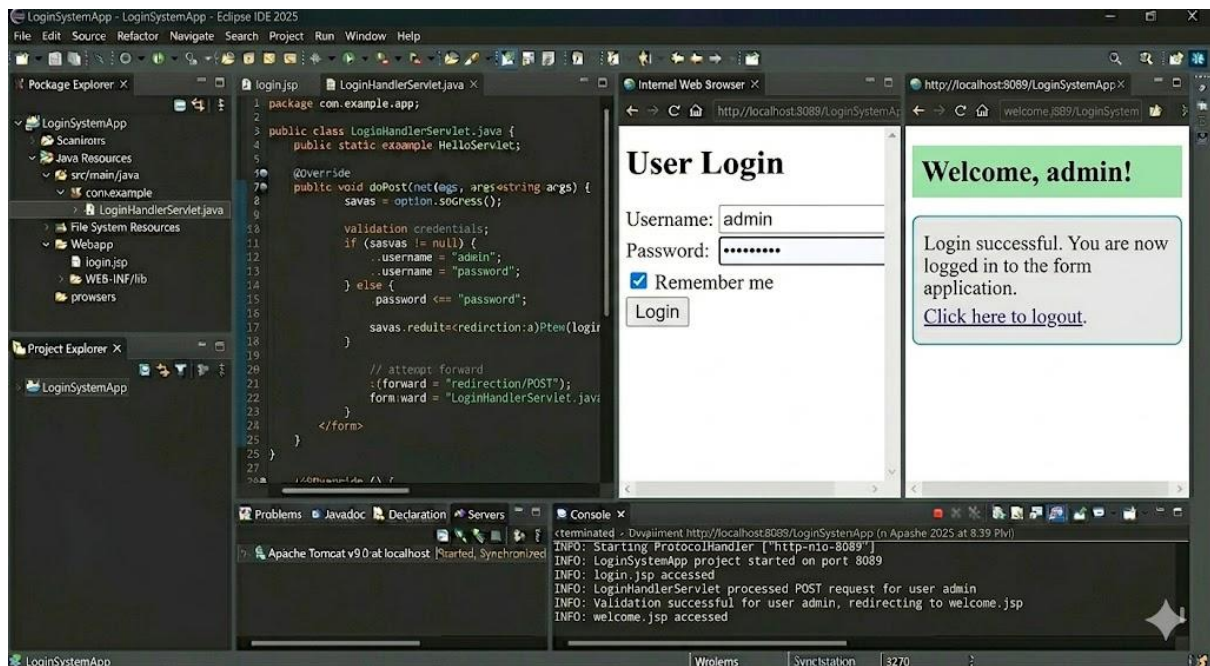
◆ Step 6: Handle Login Success

- If credentials are correct:
 - Redirect or forward to a new JSP page
 - Display welcome message
-

◆ Step 7: Handle Login Failure

- If credentials are incorrect:
 - Show error message
 - Allow user to retry
-

Output



Result

The application should:

- Accept login credentials
- Validate using Servlet
- Display success or failure response
- Allow retry on failure

Experiment 6: Student Marks Processing System using JSP and Servlet

1. Aim

To develop a **student marks processing system** where a **JSP page** accepts marks for multiple subjects, a **Servlet** calculates the total and average, and the **result is displayed** in another **JSP page** using **web.xml** configuration.

In this experiment:

1. The **input JSP page** collects student marks.
2. The **Servlet** receives the marks and performs calculations.
3. The **result JSP page** displays the total and average marks.
4. The **web.xml** file is used to configure servlet mapping.

```
StudentMarksApp
|
|-- WebContent
|   |-- index.jsp
|   |-- result.jsp
|
|-- src
|   |-- com.wtma
|       |-- MarksServlet.java
|
|-- WEB-INF
|   |-- web.xml
```

Execution Steps

1. Create a **Dynamic Web Project** in Eclipse.
2. Create the **JSP files** (index.jsp and result.jsp).
3. Create a **Servlet class** named MarksServlet.
4. Configure the servlet in **web.xml**.
5. Start the **Tomcat Server**.
6. Run the project.
7. Enter marks in the form.

8. Click **Calculate**.
9. The system displays **Total and Average Marks**.

Role of web.xml in the Student Marks Processing System

web.xml is known as the **Deployment Descriptor** of a Java Web Application. It is used to configure how different components of the web application (such as **Servlets, JSP pages, filters, and listeners**) interact with each other.

Servlet Configuration (web.xml)

```
<? XML
<web-app>

<servlet>
<servlet-name>MarksServlet</servlet-name>
<servlet-class>com.wtma.MarksServlet</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>MarksServlet</servlet-name>
<url-pattern>/marks</url-pattern>
</servlet-mapping>

</web-app>
```

In a Servlet-based web application, there are two ways to map a URL to a Servlet:

1. Using Annotation (@WebServlet)
2. Using Deployment Descriptor (web.xml)

Both methods achieve the same goal — mapping a URL request to a Servlet class — but they differ in where the configuration is defined and how the server reads it.

1. Servlet Registration

The <servlet> tag is used to declare the Servlet inside the web application.

```
<servlet>
  <servlet-name>MarksServlet</servlet-name>
  <servlet-class>com.wtma.MarksServlet</servlet-class>
</servlet>
```

Explanation

- <servlet-name>

- A logical name given to the servlet.
 - It acts as an identifier for mapping.
- **<servlet-class>**
 - Specifies the fully qualified Java class name of the Servlet.
 - In this case: **com.wtma.MarksServlet**.

This tells the Tomcat server that this class should be treated as a Servlet component.

2. URL Mapping

The <servlet-mapping> tag maps a URL pattern to the Servlet.

```
<servlet-mapping>
  <servlet-name>MarksServlet</servlet-name>
  <url-pattern>/marks</url-pattern>
</servlet-mapping>
```

Explanation

- **<servlet-name>**
 - Must match the servlet-name declared earlier.
- **<url-pattern>**
 - Defines the URL used to access the Servlet.

In this program:

/marks

means that whenever the browser sends a request to marks, the server executes MarksServlet.

3. How web.xml Works in This Program

The sequence of execution is:

1 The user opens index.jsp and enters marks.

2 The form submission is written as:

```
<form action="marks" method="post">
```

3 The request goes to the server with URL /marks.

4 The server checks web.xml and finds:

/marks → MarksServlet

5 The MarksServlet processes the marks and calculates:

- Total
- Average

6 The servlet forwards the result to:

result.jsp

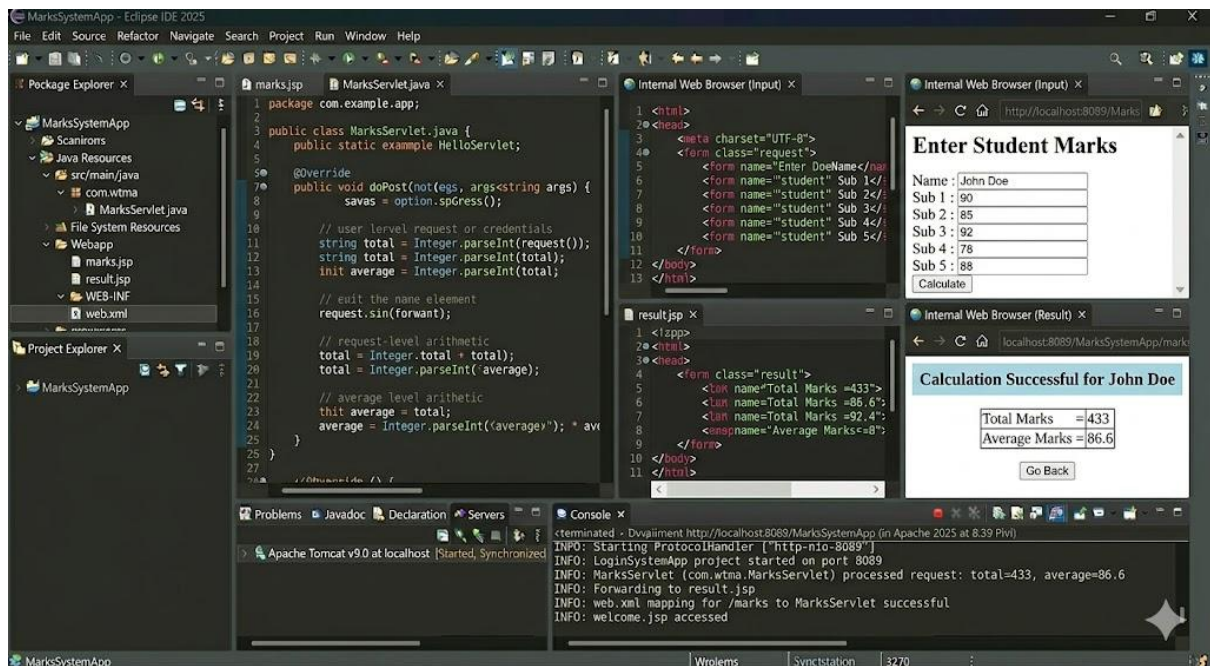
7 The result JSP page displays the output.

Algorithm

1. Start the program.
2. Create the JSP input page (marks.jsp).
 - Design an HTML form using JSP.
 - Add input fields for:
 - Student Name
 - Marks of Subject 1
 - Marks of Subject 2
 - Marks of Subject 3
 - Marks of Subject 4
 - Marks of Subject 5
 - Set the form action to the Servlet URL pattern defined in web.xml.
3. Submit the form data.
 - When the user clicks the Submit button, the marks entered in the form are sent to the Servlet using the POST method.
4. Configure the Servlet in web.xml.
 - Define the Servlet class using the <servlet> tag.
 - Map the Servlet to a URL pattern using the <servlet-mapping> tag.
5. Create the Servlet class (MarksServlet.java).

- Extend the HttpServlet class.
- Override the doPost() method to process form data.
- 6. Retrieve the form parameters.
 - Use request.getParameter() to obtain the marks entered in the JSP page.
- 7. Convert the input values.
 - Convert the retrieved marks from String to integer using Integer.parseInt().
- 8. Calculate the total marks.
 - Add the marks of all subjects to compute the total.
- 9. Calculate the average marks.
 - Divide the total marks by the number of subjects to compute the average.
- 10. Store the calculated results.
 - Set the total and average as request attributes using request.setAttribute().
- 11. Forward the request to the result JSP page.
 - Use RequestDispatcher to forward the request to result.jsp.
- 12. Display the results in result.jsp.
 - Retrieve the total and average values using request.getAttribute().
 - Display the calculated results on the webpage.
- 13. Stop the program.

Output



Result

Thus the **student marks processing system using JSP and Servlet** was successfully **implemented** and the **total and average marks** were calculated and displayed using **web.xml configuration**.

Experiment 7 : Create a servlet program that demonstrates session tracking using HttpSession by implementing a simple login system and maintaining user session information.

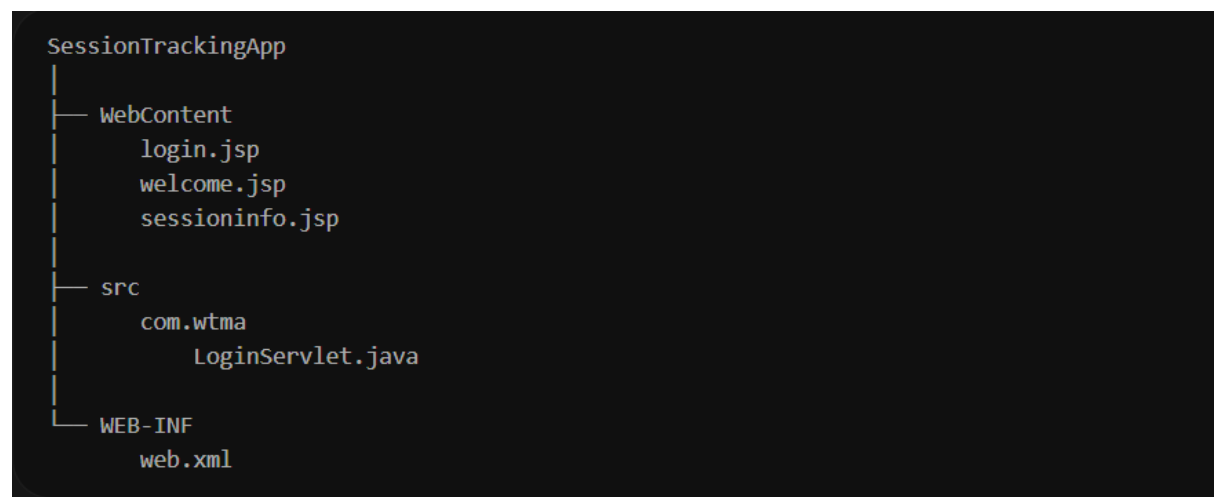
Aim

To develop a **Servlet program that demonstrates session tracking using HttpSession by implementing a simple login system and maintaining user session information.**

In this experiment:

1. User enters **username and password**.
2. Servlet validates login.
3. If valid, **HttpSession is created**.
4. Username is stored in session.
5. Welcome page displays session information and session details is displayed in other pages.

Project Structure

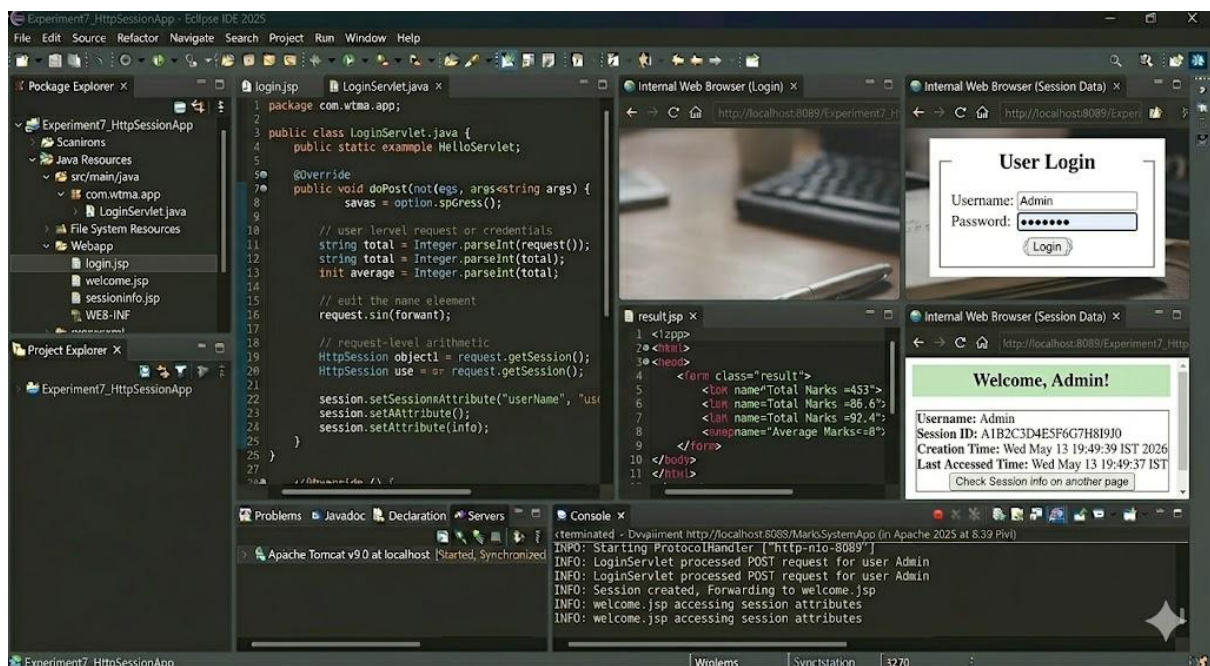


Steps

1. Start the program.
2. Display the **login page** to the user.
3. User enters **username and password**.

4. The form sends data to **LoginServlet**.
5. The servlet retrieves the login credentials using `request.getParameter()`.
6. Validate the username and password.
7. If the credentials are correct:
 - Create a session using `HttpSession session = request.getSession();`
 - Store username in session using `session.setAttribute()`.
8. Forward the request to **welcome.jsp**.
9. Display the welcome message using session data.
10. Provide a link to **Session Information Page**.
11. The session information page retrieves session attributes using `getAttribute()`.
12. Display session details such as:
 - Username
 - Session ID
 - Session creation time
13. Stop.

Output



Result

Thus the Servlet program demonstrating **session** tracking using **HttpSession** was successfully implemented and session information was maintained across multiple pages.